

Souvenir

Amaru sta comprando dei souvenir in un negozietto. Esistono N **tipi** di souvenir. Nel negozio sono disponibili illimitati souvenir di ogni tipo. Ogni tipo di souvenir ha un prezzo fisso: un souvenir del tipo i ($0 \leq i < N$) ha un prezzo di $P[i]$ monete, con $P[i]$ intero positivo.

Amaru sa che i tipi di souvenir sono ordinati per prezzo decrescente e che i prezzi dei souvenir sono distinti: $P[0] > P[1] > \dots > P[N-1] > 0$. Amaru conosce il valore di $P[0]$, ma non conosce i prezzi degli altri souvenir. Per comprare alcuni souvenir, Amaru effettuerà una serie di transazioni con il venditore. Ogni transazione consiste nelle seguenti fasi:

1. Amaru consegna al venditore un certo numero (positivo) di monete.
2. Il venditore mette queste monete in un mucchio dove Amaru non può vederle.
3. Il venditore scorre ogni tipo di souvenir $0, 1, \dots, N-1$ nell'ordine, uno alla volta.
 - Per il souvenir di tipo i , se il numero attuale di monete nel mucchio è $\geq P[i]$:
 - il venditore rimuove $P[i]$ monete dal mucchio, poi
 - mette sul tavolo un souvenir di tipo i .
 - Al massimo un souvenir di ogni tipo viene consegnato per ogni transazione.
4. Il venditore dà ad Amaru tutte le monete rimaste nel mucchio e tutti i souvenir sul tavolo.

Prima di ogni transazione, non ci sono né monete né souvenir sul tavolo. Il vostro compito è aiutare Amaru a eseguire un certo numero di transazioni, in modo che:

- in ogni transazione compri **almeno un** souvenir e
- complessivamente compri **esattamente** i souvenir del tipo i , per ogni $0 \leq i < N$.

Nota che ciò significa che Amaru non deve acquistare alcun souvenir del tipo 0. Non è necessario minimizzare il numero di transazioni e hai una disponibilità illimitata di monete.

Dettagli di implementazione

Devi implementare la seguente funzione.

```
void buy_souvenirs(int N, long long P0)
```

- N : il numero di tipi di souvenir.
- $P0$: il valore di $P[0]$.
- Questa funzione viene richiamata esattamente una volta per ogni caso di test.

Puoi effettuare chiamate alla seguente funzione per eseguire una transazione:

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- M : il numero di monete consegnate al venditore da Amaru.
- La procedura restituisce una `pair`. Il primo elemento è un array L , contenente i tipi di souvenir acquistati (in ordine crescente). Il secondo elemento è un numero intero R , il numero di monete restituite ad Amaru dopo la transazione.
- È necessario che $P[0] > M \geq P[N - 1]$. La condizione $P[0] > M$ assicura che Amaru non acquisti alcun souvenir di tipo 0 e $M \geq P[N - 1]$ assicura che Amaru acquisti almeno un souvenir. Se queste condizioni non sono soddisfatte, la soluzione riceverà il verdetto `Output isn't correct: Invalid argument`. Si noti che, contrariamente a $P[0]$, il valore di $P[N - 1]$ non viene fornito in ingresso.
- La procedura può essere chiamata al massimo 5000 volte in ogni caso di test.

Il grader **non è adattivo**. Ciò significa che la sequenza di prezzi P è fissata prima che venga chiamato `buy_souvenirs`.

Assunzioni

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$ per ogni i tale che $0 \leq i < N$.
- $P[i] > P[i + 1]$ per ogni i tale che $0 \leq i < N - 1$.

Subtask

Subtask	Punteggio	Assunzioni aggiuntive
1	4	$N = 2$
2	3	$P[i] = N - i$ per ogni i tale che $0 \leq i < N$.
3	14	$P[i] \leq P[i + 1] + 2$ per ogni i tale che $0 \leq i < N - 1$.
4	18	$N = 3$
5	28	$P[i + 1] + P[i + 2] \leq P[i]$ per ogni i tale che $0 \leq i < N - 2$. $P[i] \leq 2 \cdot P[i + 1]$ per ogni i tale che $0 \leq i < N - 1$.
6	33	Nessuna assunzione aggiuntiva.

Esempio

Si consideri la seguente chiamata.

```
buy_souvenirs(3, 4)
```

Esistono $N = 3$ tipi di souvenir e $P[0] = 4$. Ci sono solo tre possibili sequenze di prezzi P : $[4, 3, 2]$, $[4, 3, 1]$, e $[4, 2, 1]$.

Supponiamo di chiamare `transaction(2)` e che la chiamata restituisca $([2], 1)$, il che significa che Amaru ha comprato un souvenir del tipo 2 e il venditore gli ha restituito 1 moneta. Osserviamo che questo ci permette di dedurre che $P = [4, 3, 1]$, poiché:

- Per $P = [4, 3, 2]$, `transaction(2)` avrebbe restituito $([2], 0)$.
- Per $P = [4, 2, 1]$, `transaction(2)` avrebbe restituito $([1], 0)$.

Quindi si può chiamare `transaction(3)`, che restituisce $([1], 0)$, il che significa che Amaru ha comprato un souvenir del tipo 1 e il venditore gli ha restituito 0 monete. Finora, in totale, ha acquistato un souvenir del tipo 1 e un souvenir del tipo 2.

Infine, si può chiamare `transaction(1)`, che restituisce $([2], 0)$, il che significa che Amaru ha comprato un souvenir di tipo 2. Si noti che in questo caso avremmo potuto usare anche `transaction(2)`. A questo punto, in totale Amaru ha un souvenir di tipo 1 e due souvenir di tipo 2, come richiesto.

Grader di esempio

Formato di input:

```
N
P[0] P[1] ... P[N-1]
```

Formato di output:

```
Q[0] Q[1] ... Q[N-1]
```

Dove $Q[i]$ è il numero di souvenir del tipo i acquistati in totale per ogni i tale che $0 \leq i < N$.